Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

WEEK 1 - SESSION C2

Development Environment: VS Code, Node/npm, Live Server & Your First HTML File

Student Learning Material

The big question
How do I turn my computer into a proper environment where I can write, run, inspect and organize frontend code?

In C1 you learned what happens when a browser communicates with the web. C2 moves to the developer side. You will learn the tools and workflow needed to create and test frontend code.

Learning Objectives

- Understand what a code editor is and why VS Code is useful.
- Understand files, folders, extensions and basic project structure.
- Understand Node.js as a JavaScript runtime and why frontend tooling uses it.
- Understand npm, packages, package.json and node_modules.
- Use the VS Code terminal and essential commands.
- Create a frontend project folder from scratch.
- Create a valid HTML document.
- Run an HTML page locally using Live Server.
- Use browser DevTools to inspect the page.
- Recognize common environment/setup problems and debug them logically.

By the end of C2
You should be able to create a folder, open it in VS Code, create index.html, write a valid HTML document, launch it at localhost with Live Server, inspect it in the browser, change the source, save it and see the result update.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

1. What Is a Development Environment?

A development environment is the collection of software, folders, tools and configuration used to create and test software. For this course, your basic frontend environment contains your operating system, VS Code, a browser, Node.js/npm and a local development server.

Tool	Role	Why you need it
VS Code	Write and organize source code	Editing, navigation and development features.
Browser	Runs/displays frontend code	HTML rendering and DevTools.
Node.js	Runs JavaScript outside the browser	Modern frontend tooling.
npm	Manages JavaScript packages/scripts	Installs tools and dependencies.
Live Server	Serves local files over HTTP	Convenient localhost workflow.

Mental model
VS Code is where you write. Node/npm provide the tooling ecosystem. The browser runs your frontend. Live Server gives the browser a local HTTP address.

2. VS Code: Your Main Workspace

Visual Studio Code is a source-code editor. It is not the browser and it is not Node.js. Think of it as your main workbench.

Area	Useful for
Explorer	Browse project files/folders.
Editor	Write and edit code.
Terminal	Run commands inside VS Code.
Source Control	Inspect Git changes.
Extensions	Add development features.
Problems	See detected issues when supported.
Command Palette	Find VS Code commands quickly.

Action	Shortcut
Save	Ctrl + S
Open folder	Ctrl + K, Ctrl + O
Terminal	Ctrl + `
Command Palette	Ctrl + Shift + P
Search files	Ctrl + Shift + F
Quick open	Ctrl + P

Good habit
Open the project folder, not only one file. Frontend applications are collections of related files.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

3. Files, Folders and Extensions

Even a small website can contain HTML, CSS, JavaScript, images, fonts and configuration.

```
my-first-site/
├── index.html
├── css/
│   └── style.css
├── js/
│   └── script.js
└── images/
    └── logo.png
```

Extension	Typical purpose
.html	HTML structure
.css	Styles and layout
.js	JavaScript
.json	Structured data/configuration
.png/.jpg/.webp/.svg	Images
.md	Documentation
.gitignore	Files Git should ignore

4. The Terminal

The terminal lets you control your computer by typing commands. You do not need to become a command-line expert, but you should know a small set.

Command	Purpose	Example
pwd	Show current directory	pwd
ls	List files/folders on macOS/Linux	ls
dir	List files/folders in Windows Command Prompt	dir
cd	Move into a folder	cd my-first-site
cd ..	Move to parent folder	cd ..
mkdir	Create a folder	mkdir my-first-site
cls / clear	Clear terminal	cls
code .	Open current folder in VS Code	code .

Windows note
PowerShell supports many familiar commands, but behavior can differ from Linux/macOS. Learn the commands you actually use rather than memorizing dozens.

5. What Is Node.js?

Node.js is a JavaScript runtime. It allows JavaScript to execute outside the browser and is widely used for frontend development tooling.

Browser JavaScript	Node.js
Runs inside a browser	Runs as a separate process

Has browser APIs such as document/window	Has Node-specific APIs
Used for UI and browser interaction	Used for tooling, scripts and servers
Can manipulate the DOM	Does not automatically provide a browser DOM

Important
Node.js is not Chrome. document.querySelector() requires a browser DOM; Node.js alone does not provide the same document object.

Why frontend developers need Node

- Modern frontend tools are distributed through the JavaScript package ecosystem.
- npm scripts start development servers and build applications.
- Vite and React projects use Node-based tooling.
- Later course topics depend on this environment.

6. npm: The JavaScript Package Manager

npm is commonly used to install JavaScript packages and run project scripts. A package is reusable software that can be installed into a project.

Term	Meaning
Package	Reusable software distributed for installation.
package.json	Project manifest containing metadata, scripts and dependencies.
node_modules	Local directory containing installed packages.
package-lock.json	Records resolved dependency information for reproducible installs.

Command	Purpose
npm --version	Check npm version.
npm init	Create package.json interactively.
npm init -y	Create package.json with defaults.
npm install	Install dependencies.
npm install <package>	Install a package.
npm install -D <package>	Install a development dependency.
npm run <script>	Run a package.json script.

Core idea
package.json describes the project. npm manages packages and scripts. node_modules contains installed packages.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

7. Understanding package.json

```
{
  "name": "my-first-project",
  "version": "1.0.0",
  "scripts": {
    "dev": "..."
  },
  "dependencies": {
    "some-package": "..."
  }
}
```

Field	Meaning
name	Project/package name.
version	Project/package version.
scripts	Commands available through npm run.
dependencies	Packages required by the application.
devDependencies	Packages mainly needed during development/building.

Do not edit blindly
If you are unsure what a package or script does, inspect the documentation before deleting or changing it.

8. Why Use a Local Server?

A simple HTML file can sometimes be opened with file://. However, frontend development is closer to real web behavior when files are served through HTTP from localhost.

```
file://...
  versus
http://localhost:5500/...
```

Direct file	Local server
file:// URL	http://localhost:PORT
Fine for simple experiments	Closer to real web behavior
Some features can behave differently	Provides an HTTP origin
Limited local request behavior	Supports local HTTP requests

Live Server is a VS Code extension that serves your project locally and can reload the page after changes.

1. Open the project folder in VS Code.
2. Install Live Server if needed.
3. Open index.html.
4. Launch Live Server.
5. Confirm the browser opens a localhost URL.
6. Edit and save the file.
7. Observe the browser update.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

Remember
localhost refers to your own computer. 127.0.0.1 is another common way to refer to the local machine.

9. Your First HTML Document

HTML stands for HyperText Markup Language. It describes the structure and meaning of web content.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, Web!</h1>
  <p>I am learning frontend development.</p>
</body>
</html>
```

Element	Purpose
<!DOCTYPE html>	Declares modern HTML syntax.
<html lang="en">	Root element; declares document language.
<head>	Metadata and supporting resources.
<meta charset="UTF-8">	Declares character encoding.
<meta name="viewport" ...>	Helps mobile viewport behavior.
<title>	Browser tab/document title.
<body>	Visible document content.
<h1>	Main heading.
	Paragraph.

10. Elements, Tags and Attributes

An element is a complete HTML construct. A common element has an opening tag, content and a closing tag.

```
<p>Hello students.</p>
```

Term	Example	Meaning
Opening tag		Starts an element.
Content	Hello students.	Content inside the element.
Closing tag		Ends an element.
Element	Hello students.	Complete HTML construct.
Attribute	class="intro"	Additional element information.

```
<p class="intro">Welcome to CSE-2340.</p>
```

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

11. HTML Structure and Nesting

```
<body>
  <main>
    <h1>Frontend Development</h1>
    <p>Learn the web step by step.</p>
  </main>
</body>
```

- <main> is a child of <body>.
- <h1> and <p> are children of <main>.
- <h1> and <p> are siblings because they share the same parent.

Indentation does not create nesting, but clean indentation makes your code much easier to understand and debug.

12. Guided Lab: Build Your First Page

1. Create a folder named cse-2340-week1-c2.
2. Open that folder in VS Code.
3. Create a file named index.html.
4. Add the HTML5 boilerplate from this material.
5. Change the title and heading to your own content.
6. Launch the page with Live Server.
7. Add another paragraph and a secondary heading.
8. Save the file and observe the browser update.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My CSE-2340 Page</title>
</head>
<body>
  <h1>Welcome to My First Page</h1>
  <p>This page was created during Week 1, Session C2.</p>
</body>
</html>
```

13. Browser DevTools: Your First Inspection

Panel	**Today's purpose**
Elements	Inspect the DOM and HTML.
Console	See JavaScript messages/errors later.
Network	Inspect HTTP requests; connects to C1.
Sources	Inspect loaded source files.

1. Open your page.
2. Open DevTools.
3. Select Elements.

4. Locate the h1 element.
5. Temporarily change its text in DevTools.
6. Refresh the page.
7. Notice the temporary change disappears.

Important lesson
Changing the DOM through DevTools does not edit index.html. Changing and saving index.html changes your source code.

14. Source HTML vs DOM

The HTML file is your source document. The DOM is the browser's in-memory representation. JavaScript can later modify the DOM without changing the original HTML file.

```
Source:
<h1>Hello</h1>

JavaScript:
document.querySelector("h1").textContent =
  "Hello, CSE-2340!";
```

Connection to C1
C1 taught HTML → DOM → rendering. C2 gives you the practical workflow for creating the HTML that enters that pipeline.

15. Verify Your Environment

```
node --version
npm --version
```

Check	Expected result
VS Code	Can open a project folder and edit files.
Browser	Can open localhost and DevTools.
node --version	Displays a Node version.
npm --version	Displays an npm version.
Live Server	Opens your HTML at localhost.
index.html	Loads correctly.

If 'node' is not recognized
Check that Node.js is installed, reopen the terminal after installation, and verify the system PATH before reinstalling everything.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

16. What Is PATH? (Only What You Need)

PATH is an operating-system environment variable containing locations where executable programs can be found. When you type node, the operating system searches those locations for the executable.

node --version
↓
operating system checks PATH
↓
finds Node executable
↓
Node runs
↓
version appears

17. What Happens When npm Installs a Package?

npm install some-package
↓
package registry
↓
download package
↓
node_modules/
↓
package.json + package-lock.json updated

- package.json records the dependency.
- node_modules contains installed package files.
- package-lock.json records resolved dependency versions.
- node_modules is normally excluded from Git.
- Later you will use npm for Vite, React and other tools.

18. Common Beginner Problems

Problem	**Likely cause**	**First check**
index.html not found	Wrong filename/path	Check exact spelling and Explorer.
Page blank	Invalid/empty structure or another issue	Inspect Elements and source.
Live Server fails	Extension/workspace issue	Open the project folder and check Live Server.
node not recognized	Node/PATH/terminal issue	Run node --version in a new terminal.
npm not recognized	Node/npm/PATH issue	Run npm --version.
Changes do not appear	File not saved or wrong page	Ctrl+S and verify URL.
index.html.txt	Hidden file extension	Show file extensions and rename.
Old page displayed	Cache/wrong local page	Hard reload and verify URL.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

Debugging rule
Separate environment problems from code problems. Ask: Is the tool working? Is my file correct? Is the browser loading the correct file?

19. Clean Starter Project Structure

```
my-frontend-project/
├── index.html
├── css/
│   └── style.css
├── js/
│   └── script.js
├── images/
│   └── ...
└── README.md
```

This is intentionally simple. Do not create dozens of folders before you need them.

20. Terms You Must Not Mix Up

Term	Correct meaning
VS Code	Code editor/development workspace.
Browser	Client software that loads and renders web applications.
Node.js	JavaScript runtime outside the browser.
npm	Package manager/CLI in the JavaScript ecosystem.
Package	Reusable software that can be installed.
package.json	Project manifest containing scripts/dependencies.
node_modules	Local directory containing installed packages.
Live Server	Local development server workflow.
localhost	Hostname referring to the local computer.
HTML	Markup language describing document structure.
DOM	Browser's in-memory document representation.
DevTools	Browser tools for inspecting/debugging web pages.

21. In-Class Activities

Activity A - Environment check

- Open VS Code.
- Open a project folder.
- Open the terminal.
- Run node --version.
- Run npm --version.
- Create index.html.
- Run it with Live Server.

Mahir Shadid
Lecturer, CSE, IIUC
Coordinator, CSE-2340

Activity B - DOM inspection

Use DevTools → Elements to identify html, head, body, h1 and p. Explain their parent-child relationships.

Activity C - Controlled break

Intentionally make one small HTML mistake, observe what happens, then fix it. The goal is to become comfortable debugging rather than fearing errors.

22. Practice Challenge: Personal Introduction Page

Create a page containing:

- A meaningful title.
- One main h1.
- At least two paragraphs.
- A secondary heading.
- A list of three items.
- A link to a website.
- An image if available.
- Clean indentation.
- A valid HTML5 document structure.
- Successful execution through Live Server.

23. Homework / Self-Practice

1. Install and verify VS Code, Node.js and npm.
2. Create cse-2340-week1-c2.
3. Create index.html and build a personal introduction page.
4. Run it through Live Server.
5. Inspect the DOM using DevTools.
6. Take a screenshot of the page and another of the Elements panel.
7. Write five sentences explaining VS Code, Node.js, npm, Live Server and the browser.
8. Explain why localhost differs from a public deployed website.

24. Essential Glossary

Term	Student-friendly definition
Code editor	Software designed for writing and managing source code.
Development environment	Tools and setup used to build and test software.
Terminal	Text-based interface for running commands.
Runtime	Environment that executes program code.
Node.js	JavaScript runtime used widely for development tooling.
npm	Package manager/CLI for the JavaScript ecosystem.
Package	Reusable software that can be installed.
Dependency	A package a project relies on.
package.json	Project manifest containing scripts, dependencies and metadata.
node_modules	Local folder containing installed npm packages.

Live Server	Local server workflow for serving web files.
localhost	Hostname referring to the local computer.
HTML	Markup used to structure web documents.
Element	A complete HTML construct.
Tag	Markup syntax used to start/end an HTML element.
Attribute	Additional information attached to an HTML element.
DOM	Browser's object representation of the document.
DevTools	Browser tools for inspecting and debugging web applications.

25. The One Mental Model to Keep

```
YOUR COMPUTER
│
├── VS Code
│     └── write and organize source code
│
├── Terminal
│     └── run commands
│
├── Node.js
│     └── run JavaScript-based tooling
│
├── npm
│     └── manage packages and scripts
│
└── Browser
      ├── load the page
      ├── build the DOM
      ├── apply CSS
      ├── run browser JavaScript
      └── render the interface

Live Server
    ↓
http://localhost:PORT
    ↓
Browser loads your project
```

If you remember only one thing
Write code → serve/build it with development tools → open it in the browser → inspect the result → change the source → test again.

26. Final Self-Check

- I can open a project folder in VS Code.
- I can use the integrated terminal.
- I can verify Node.js and npm.
- I can explain package.json and node_modules.

- I can create a valid index.html.
- I can launch my page through Live Server.
- I understand localhost.
- I can inspect my HTML in DevTools.
- I can explain source HTML versus the DOM.
- I can distinguish an environment problem from a code problem.

27. Connection to the Next Sessions

C2 gives you the environment. C3 builds on it with semantic HTML: headings, text, links, images, forms and document structure. Later, the same workflow will be used for CSS, JavaScript, DOM manipulation, APIs, React, Firebase and deployment.